

Modeling Deception in Multi-Robot Target-Attacker-Defender Game via Deep Reinforcement Learning

Fandi Gou^{1,2}, Chenyu Zhao^{1,2}, Haikuo Du^{1,2} and Yunze Cai^{1,2,3,4}

Abstract—Deception is a crucial strategy in adversarial scenarios, yet its application in multi-agent confrontations remains understudied. This paper investigates deception in a multi-robot Target-Attacker-Defender (MR-TAD) game, where Attackers aim to capture Targets while evading Defenders. To model deception effectively, we propose a hierarchical decision-making framework that integrates multi-agent reinforcement learning (MARL) for high-level deceptive strategies and optimal control for low-level motion control. Furthermore, we introduce a novel composite deception-oriented reward function, which combines hitting rewards, belief switch rewards, and position advantage rewards to facilitate the training of deceptive behaviors. Simulation results across varying numbers of robots demonstrate that incorporating deception significantly increases the success rate of Attackers, with an average improvement of over 70% compared to non-deceptive strategies. Additionally, real-world experiments with omnidirectional mobile robots further confirm the effectiveness of the proposed method. This study establishes a generalizable framework for modeling deception in multi-agent systems, with potential applications in various multi-agent scenarios.

I. INTRODUCTION

Deception is a sophisticated form of intelligence that is prevalent across various domains, such as animal predation [1], cyber attacks [2], and robotics [3]. These deceptive behaviors often involve concealing true intentions to devise intricate strategies. In adversarial contexts, such strategies can effectively mislead opponents regarding the deceivers' true objectives, thereby increasing the likelihood of achieving their goals. A classic example is the cooperative hunting behavior of wolf packs, where some wolves feign attacks to drive the prey in a specific direction, while others lie in ambush along the prey's escape route, ultimately capturing the prey successfully. With the growing interest in swarm intelligence in recent years, applying the concept of deception to swarm robotic confrontations holds significant potential for enhancing combat tactics and strategic efficiency.

The study of deception has been extensively explored in adversarial games [4]. Ornik and Topcu [5] introduced a mathematically rigorous framework for deception within the

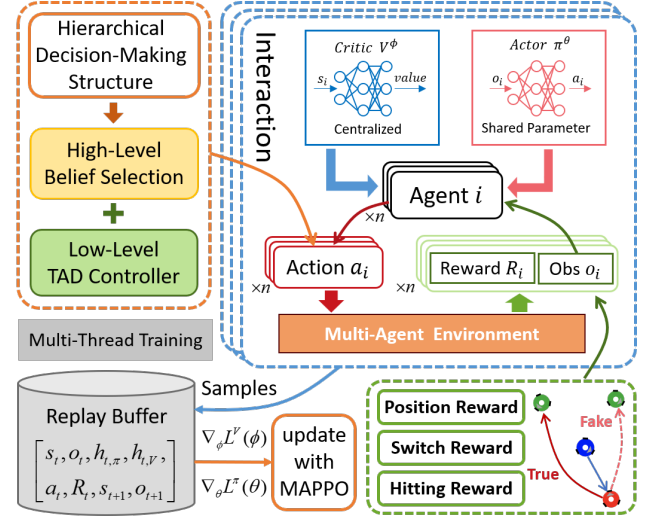


Fig. 1: Overall structure of deception in multi-robot Target-Attacker-Defender game.

context of optimal control, and their subsequent research [6] further investigated supervisory control under this framework using reinforcement learning. In both works, adversarial games are formulated with discrete state and action spaces. Huang *et al.* [7] proposed a dynamic game framework for rational and persistent robot deception in multi-agent systems, modeling the interaction as an N-player K-stage game and employing Perfect Bayesian Nash Equilibrium (PBNE) to predict players' actions and beliefs. However, this framework suffers from high computational complexity in large-scale systems, and updating belief dynamics via PBNE remains challenging in complex tasks.

Asgharnia *et al.* conducted extensive research on deception in differential games [8]–[10]. In [8], they proposed a hierarchical policy system for a continuous-domain game of guarding multiple static territories, where an invader employs deceptive strategies to mislead defenders. A follow-up study [9] introduced a grid-world version of the game, leveraging minimax and Q-learning at different policy levels to improve invader's success rate. Later work [10] integrated reinforcement learning to optimize multi-objective deception strategies in continuous domains, further improving the invader's payoff. Despite these advancements, the proposed approaches remain computationally intensive and are limited to scenarios with only one single invader or defender.

Most existing studies on deception either focus on scenarios with a small number of agents or operate within

*This work was supported by Aeronautical Science Foundation of China (No.20220001057001 and No.20240001057002), and the open project of the National Key Laboratory of Air-based Information Perception and Fusion under Grant (202437).

All authors are with the ¹ School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, Shanghai 200240, and ² Key Laboratory of System Control and Information Processing, Ministry of Education of China, Shanghai 200240 (email: finley-gou3@sjtu.edu.cn, pxwzq6@sjtu.edu.cn, duhaikuo1013@sjtu.edu.cn, yzcai@sjtu.edu.cn).

³ State Key Laboratory of Submarine Geoscience, School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, Shanghai 200240.

⁴National Key Laboratory of Air-based Information Perception and Fusion, Henan Luoyang, 471000.

discrete state and action spaces. These limitations restrict the applicability of deception strategies in large-scale, continuous game settings, which more accurately reflect real-world situations. Therefore, investigating deception in a multiplayer setting with continuous domains is essential. The multi-robot Target-Attacker-Defender (MR-TAD) game provides a suitable framework for this exploration. In this scenario, the target is a natural focal point for the attacker to employ deception against defenders, and the game contains all the necessary elements to study deception effectively.

The Target-Attacker-Defender (TAD) game is an extension of the well-studied pursuit-evasion game [11]. In a TAD game, the Attacker aims to capture the Target while avoiding capture by the Defender, whereas the Defender seeks to protect the Target from the Attacker while simultaneously attempting to intercept the Attacker. Based on differential game theory, policies in the TAD game are typically derived through solving Hamilton-Jacobi-Bellman (HJB) equations. Existing research has explored various aspects of the TAD game, including the regions of advantage for each party [12], criteria for role switching [13], and optimal pursuit-evasion strategies [14].

Despite the growing interest in TAD games, relatively few studies have examined them in a multiplayer setting. Studies [15], [16] have considered multiplayer scenarios, but only by scaling up a single party to multiplayer. Work [17] investigated the TAD game with multiple Targets, Attackers, and Defenders by decomposing it into different independent single TAD games, where each sub-TAD group operated without interference from others. However, this approach restricts coordination and intelligent decision-making among agents, as interactions between sub-TAD groups are not considered.

Modeling deception in an MR-TAD game presents significant challenges due to the complexity of strategic interactions and the need for adaptive decision-making. Traditional approaches struggle to capture the dynamic and multi-agent nature of deception, making artificial intelligence (AI) a crucial method for addressing this problem. Among AI techniques, multi-agent reinforcement learning (MARL) is particularly well-suited due to its ability to optimize sequential decision-making in complex environments. MARL has been widely applied in multi-agent systems, especially in hierarchical decision-making frameworks where high-level strategies guide low-level control [18]. Such frameworks have proven effective in tasks like aerial combat [19] and robot navigation [20]. Inspired by this, we propose a MARL-based decision-making structure for deception in MR-TAD games, where MARL decides the high-level belief selection while TAD controllers handle the low-level control. The overall framework is illustrated in Fig. 1, and our main contributions are summarized as follows.

- 1) We formulated a problem of MR-TAD games where agents can switch their goals, and a novel hierarchical decision-making scheme is proposed to model deception in MR-TAD games.
- 2) A composite deception-oriented reward design method

is presented to facilitate the training of deceptive behaviors, greatly improving training efficiency and convergence.

- 3) Simulations of varying numbers of robots are carried out to demonstrate the scalability and robustness of our method. Besides, we conducted real-world experiments based on omnidirectional vehicles.

The remainder of this article is organized as follows. Section II presents the preliminaries and problem formulation of the MR-TAD game. Section III details the methodology, including the hierarchical framework, reward function design, and network structure. Section IV discusses simulation results and experiments under various settings. Finally, Section V concludes the article.

II. PRELIMINARIES AND PROBLEM FORMULATION

A. Multi-Agent Reinforcement Learning

Multi-agent reinforcement learning is a powerful tool for solving sequential decision-making problems. It is typically modeled as a Markov decision process (MDP), defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma \rangle$, where $\mathcal{S}$ is the state space, and $\mathcal{A} = [\mathcal{A}_1, \dots, \mathcal{A}_n]$ represents the joint action space, with each agent $i \in \{1, \dots, n\}$ selecting an action $a_i \in \mathcal{A}_i$. The state transition function is given by $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \mapsto [0, 1]$, and the reward function is defined as $R : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$. The discount factor $\gamma \in [0, 1]$ determines the significance of future rewards. The objective of the agents is to learn an optimal policy π that maximizes the expected discounted cumulative reward:

$$J(\pi) = E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R^t(s^t, a_1^t, \dots, a_n^t) \right] \quad (1)$$

Multi-agent proximal policy optimization (MAPPO) is a policy gradient-based approach, known for its superior convergence, optimality, and sampling efficiency compared to other MARL algorithms [21]. It has become the most popular benchmark algorithm of DRL and has a wide application in multi-agent systems. MAPPO follows the Actor-Critic structure, where the policy network π_{θ} is parameterized by θ , and the value network V_{ϕ} is parameterized by ϕ . To enhance sample efficiency during parameter updates, the importance sampling ratio $l_t(\theta)$ is introduced:

$$l_t(\theta) = \frac{\pi_{\theta}(a^t | s^t)}{\pi_{\theta_{\text{old}}}(a^t | s^t)} \quad (2)$$

where $l_t(\theta)$ denotes the probability ratio and θ_{old} is the policy parameter before the update. In this article, we select a variant of MAPPO with recurrent layers, rMAPPO, as the basic training algorithm.

B. Multi-Robot Target Defense Game

The 2D scenario of a TAD game is illustrated in Fig. 2, where the subscripts t , a , and d represent the Target, Attacker, and Defender, respectively. The robots are modeled as circular entities, with their relative distance denoted by r . The variables θ and q represent the velocity direction and

line-of-sight (LOS) angle, respectively. In this article, each robot follows a unicycle model defined as follows:

$$\begin{cases} \dot{x}_i = V_i \cos \theta_i \\ \dot{y}_i = V_i \sin \theta_i \\ \dot{\theta}_i = \omega_i \end{cases} \quad (3)$$

which means the speed of robot i remains constant while its angular velocity ω_i serves as the control input.

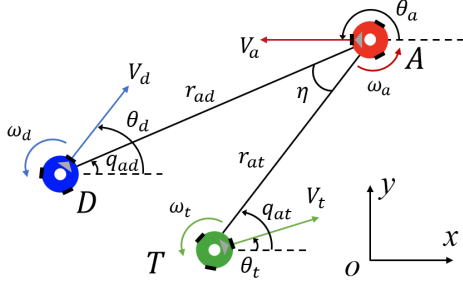


Fig. 2: Illustration of Target-Attacker-Defender game.

As shown in Fig. 2, the relative kinematics between Attacker and Target are described by the following equations:

$$\begin{cases} r_{at}\dot{q}_{at} = V_a \sin(q_{at} - \theta_a) - V_t \sin(q_{at} - \theta_t) \\ \dot{r}_{at} = V_t \cos(q_{at} - \theta_t) - V_a \cos(q_{at} - \theta_a) \end{cases} \quad (4)$$

The kinematics between Defender and Attacker can be obtained in a similar way.

The most effective policy for an Attacker to chase a Target is proportional navigation guidance (PNG), which is widely used in missile and surface vehicle control. With PNG, the velocity vector of the Attacker rotates proportionally to its LOS. Typically, PNG computes the acceleration of the robot, from which the angular velocity can be determined as:

$$\omega_a = \frac{-\kappa \dot{r}_{at} \dot{q}_{at}}{V_a} \quad (5)$$

where κ is the guidance coefficient.

In an MR-TAD game, the numbers of three entities are denoted as N_t , N_a , and N_d . To better demonstrate the effectiveness of deception, we set $N_t \geq N_d \geq N_a$, ensuring that the Target-Defender coalition has a numerical advantage. This setup highlights the impact of deception for the Attackers since they are faced with more defenders. Each Attacker maintains a true belief B_i , representing the Target it is aiming for, and these beliefs differ among Attackers. Meanwhile, each Attacker also has a fake belief $\bar{B}_i$ which guides its behavior. For Targets and Defenders, a task assignment process determines which Attacker each Defender should intercept, with the assignment updated at fixed time intervals.

C. Strategy of Target-Defender Coalitions

In a TAD game, the Targets and Defenders are on the same side, and their policies are typically calculated cooperatively together. Once the Attackers have selected their respective

Targets, it results in N_a distinct (T_i, A_i) pairs, where $i \in \{1, 2, \dots, N_a\}$. Subsequently, the Defenders assign themselves to Attackers from these pairs based on the following assignment problem:

$$\begin{aligned} \max \quad & \sum_{i=1}^{N_d} \sum_{j=1}^{N_a} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^{N_d} x_{ij} \geq 1, \quad j \in \{1, 2, \dots, N_a\} \\ & \sum_{j=1}^{N_a} x_{ij} = 1, \quad i \in \{1, 2, \dots, N_d\} \\ & x_{ij} \in \{0, 1\}, \quad \forall i, j \end{aligned} \quad (6)$$

where $c_{ij} \in C$ represents an element of the payoff matrix C , indicating the utility of each assignment solution. Specifically, c_{ij} denotes the total energy consumption of Defender i and Target j over the next time span. x_{ij} is a binary variable indicating the assignment, and this assignment problem can be solved using fundamental integer programming methods. Thus far, we have derived the primary TAD groups.

The remaining Targets select the nearest Defender from the existing TAD groups to form a secondary TAD combination. For the cooperative control law of the Target-Defender coalitions, we design the following objective function:

$$J = \frac{k_1}{2} q_{ad}^2(t_f) + \frac{k_2}{2} \dot{q}_{ad}^2(t_f) + \frac{1}{2} \int_{t_0}^{t_f} (k_3 \omega_d^2 + k_4 \omega_t^2) d\tau \quad (7)$$

where $k_1 \sim k_4$ are positive penalty parameters. By applying the concept in [22] to solve the optimal control problem, we obtain the closed-loop cooperative control laws for both Defenders and Targets. The control law for Defenders follows the primary TAD group. At this stage, the MR-TAD game is decoupled into multiple TAD games, where group compositions can dynamically change through interaction.

III. METHODOLOGY

A. Hierarchical Objective Learning

As discussed in Section II, the low-level controller for the Attacker is based on the PNG guidance law. Regarding high-level decision-making, we focus on determining the optimal timing and selection of deceptive Targets to maximize the overall success rate of the attack. Before the game begins, each Attacker is assigned a unique true belief B_i , ensuring that the true Targets of different Attackers do not overlap. Once the game starts, each Attacker holds a fake belief $\bar{B}_i$ and uses the low-level controller to move toward the fake Target $\bar{B}_i$. Both B_i and $\bar{B}_i$ belong to the Target index set $\{1, 2, \dots, N_t\}$. We then define the action space for high-level decision of the Attackers as follows:

$$a_i = [\bar{B}_i, \text{lock}] \quad (8)$$

where lock is a binary flag that indicates whether the Attacker locks its belief. Once an Attacker locks its belief, the fake belief $\bar{B}_i$ is set to the true belief B_i and remains

Algorithm 1 Deception-oriented Hierarchical Decision-Making Framework in MR-TAD Games.

Require: Observation and state of robots, policy network π_θ for Attackers.

Output: The assignment and control for each robot.

- 1: Assign the true Targets B_i for Attackers according to initial states.
 - 2: Calculate the control command for Attackers. The high-level decision $\bar{B}_i$ is generated via policy network π_θ . The low-level control ω_i is computed by proportional navigation guidance law (5).
 - 3: Formulate TAD groups through integer programming by solving equation (6).
 - 4: Calculate the control command for Targets and Defenders through optimal control (7).
 - 5: Train the policy network through rMAPPO or other MARL method.
-

unchanged for the rest of the game. The action a_i is a two-dimensional discrete vector.

Unlike traditional deep reinforcement learning methods, where actions directly control robot movements, our hierarchical structure outputs high-level decisions as actions. Since the high-level policy focuses more on long-term objectives, its control frequency should be lower than that of the low-level controller. Therefore, the high-level decision-making process operates at 10 Hz, while the PNG controller runs at 50 Hz. During the actual deployment, it is necessary to align the timestamps.

It is important to note that both the Target and Defender are aware of the Attacker's fake belief, and the Attacker's behavior follows its fake belief $\bar{B}_i$. When there is a change in an Attacker's belief or when any robot is done (e.g., shot down), the assignment of Target-Defender pairs is immediately triggered by solving (6). In all other cases, re-assignment occurs at fixed time intervals. The comprehensive hierarchical decision-making process for an MR-TAD game is presented in algorithm (1).

B. Deception-Oriented Reward Design

The design of the reward function is crucial for modeling deception, as quantifying the effectiveness of deceptive behaviors is challenging. In our method, the total reward consists of three main components: the hitting reward R_{hit} , the belief switch reward R_{swt} , and the position advantage reward R_{pos} .

1) *Hitting Reward:* The hitting reward is a direct measure of task performance for the Attackers and is defined as follows:

$$R_{\text{hit}} = \begin{cases} 100 & , \text{Attacker hits the true Target} \\ -20 & , \text{Attacker is intercepted by Defender} \end{cases} \quad (9)$$

An Attacker receives a positive reward at a given time step if the distance between the Attacker and its true Target is

smaller than the sum of their radii. Conversely, the Attacker receives a negative reward if it is intercepted by a Defender.

2) *Belief Switch Reward:* To achieve successful deception, an Attacker cannot change its belief too frequently, as this may reduce effectiveness. At the same time, holding a belief for too long is also undesirable. To balance these factors, we design the belief switch reward to regulate belief changes appropriately.

First, we record the last time an Attacker changed its belief, denoted as t_s , and compute the time interval since that change: $\Delta t = t - t_s$. When the Attacker updates its belief, if Δt is too short, it incurs a penalty for changing belief too frequently. Conversely, if Δt is too long and the Attacker has not updated its belief, it also receives a penalty to encourage effective deception. Based on this idea, we design a monotonic function of Δt to determine the belief switch reward R_{swt} :

$$R_{\text{swt}} = \begin{cases} 13.2 \ln(\Delta t + 13.2) - 44, & \text{if } \bar{B}_i(t_k) \neq \bar{B}_i(t_{k-1}) \\ -6.6 \ln(\Delta t + 13.2) + 22, & \text{otherwise.} \end{cases} \quad (10)$$

where the condition $\bar{B}_i(t_k) \neq \bar{B}_i(t_{k-1})$ indicates the switch in belief.

Besides, when the Attacker locks its belief, it incurs a penalty to discourage early locking, similar to the high cost of launching a missile [19]. This penalty gradually decreases over time, as locking in the later stages becomes necessary. We define the locking reward as follows:

$$R_{\text{lock}} = -50e^{-t} \quad (11)$$

This reward is added to R_{swt} whenever the Attacker locks its belief.

3) *Position Advantage Reward:* This reward evaluates whether the Attacker holds a positional advantage over its Defender and the true Target. Inspired by [12], we use the relative angle η between the vectors $\overrightarrow{AT}$ and $\overrightarrow{AD}$ (where T represents the true Target B_i), as illustrated in Fig. 2, to quantify positional advantage. The position reward is defined as follows:

$$R_{\text{pos}} = \begin{cases} 2, & \text{if } \cos(\eta) < 0.3 \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

As discussed in [12], when η is larger, the Defender is less likely to intercept the Attacker, according to the theory of Apollonius circle.

C. Observation Space and Network Structure

The observation vector for each Attacker consists of four components: the self-state S_{atk} , the belief state S_{blf} , the state of all Targets S_{tar} , and the state of all Defenders S_{def} . Among them, the self-state $S_{\text{atk}} = [p_i^a, v_i^a, B_i, \bar{B}_i]^\top$ describes the state of Attacker i . The belief state $S_{\text{blf}} = [B_1, B_2, \dots, B_{N_a}]^\top$ contains the true beliefs of all other Attackers. The Target state $S_{\text{tar}} = [p_1^t, v_1^t, \dots, p_{N_t}^t, v_{N_t}^t]^\top$ represents the positions and velocities of all Targets. And the Defender state, $S_{\text{def}} = [p_1^d, v_1^d, \dots, p_{N_d}^d, v_{N_d}^d]^\top$, contains

the positions and velocities of all Defenders. Finally, a concatenated state S^i for Attacker i is derived:

$$S^i = \left[(S_{\text{atk}})^\top, (S_{\text{blf}})^\top, (S_{\text{tar}})^\top, (S_{\text{def}})^\top \right]^\top \quad (13)$$

Given that MAPPO is under centralized training and decentralized execution (CTDE) framework [23], each Attacker has an actor network to map the observed states into its actions, or its fake belief. A centralized critic network is utilized to evaluate the overall state of the system. Both networks are composed of multilayer perceptrons (MLP) and recurrent neural networks (RNN), where hidden states $h^{\pi, V}$ in RNN restore the historical information to enhance decision-making [24]. Besides, all the activation functions in the networks are *tanh*.

The actor network is designed with a categorical distribution module in its final layer, which samples beliefs using a multi-discrete type. All actor networks share the same structure and model parameters. For the critic network, the input vector consists of the concatenated states of all S^i , and the output is a single real-valued number representing the state value.

IV. EXPERIMENTS

In this section, we evaluate our algorithm through various experiments with different numbers of robots. Specifically, we analyze the effectiveness of deceptive behavior based on combat trajectories, belief switching, and reward convergence. Furthermore, we conduct a Monte Carlo test to assess the success rate of MR-TAD games. The real-world experiment is conducted based on the omnidirectional mobile robot system.

A. Experimental Settings

In our setting, the number of Attackers ranges from 1 to 3. For ease of notation, we use N_t - N_a - N_d to represent each scenario. Regarding the parameters, the weights $k_1 \sim k_4$ in the objective function are set to 0.001, 1×10^3 , 0.5, and 0.5, respectively, while the guidance coefficient is $\kappa = 4$. The maximum speeds for Targets, Attackers, and Defenders are set to 0.3 m/s, 1.0 m/s, and 0.7 m/s, respectively. Additionally, the angular velocity limit is set to 0.15 rad/s for all entities.

As for the network and training-related parameters, each episode consists of a maximum of 200 steps, with each step representing 0.1 seconds. The training process is designed to run for a total of 2.56×10^6 steps. Additionally, the experiments are conducted on a computer equipped with an Intel Xeon Platinum 8352V CPU, 256 GB of memory, and an Nvidia RTX 3090 GPU. Other parameters are presented in Table I.

TABLE I: Network and training related parameters

Name	Parameter
Actor network MLP layers	2
Actor network hidden units	128, 64
Actor network RNN hidden units	32
Critic network MLP layers	3
Critic network hidden units	256, 128, 64
Critic network RNN hidden units	32
Length of RNN data chunk	10
Learning rate	0.0002
PPO update epoch	15

B. Trajectory Analysis

We conducted simulations under four scenarios: 2T-1A-1D, 3T-2A-2D, 4T-2A-3D, and 6T-3A-4D. The robot trajectories for each scenario, along with the robot indices, are illustrated in Fig. 3. The Targets, Attackers, and Defenders are represented in green, red, and blue, respectively, and the varying shades of trajectory colors indicate the sequence of motion. As observed in the figure, in all scenarios the Attackers successfully reached their true targets without being intercepted by the Defenders. Additionally, the Defenders' trajectories were influenced by the Attackers' fake beliefs, ultimately leading to failure in interception.

Besides, we have performed real-world experiments using four-wheeled omnidirectional vehicles. To simplify the experimental setup, we employed an FZMotion motion capture system to obtain vehicle states. This system accurately tracks the reflective balls that are attached to the robots and is capable of measuring positions to within 0.01mm at a frame rate of 210 frames per second, satisfying the control frequency of both levels.

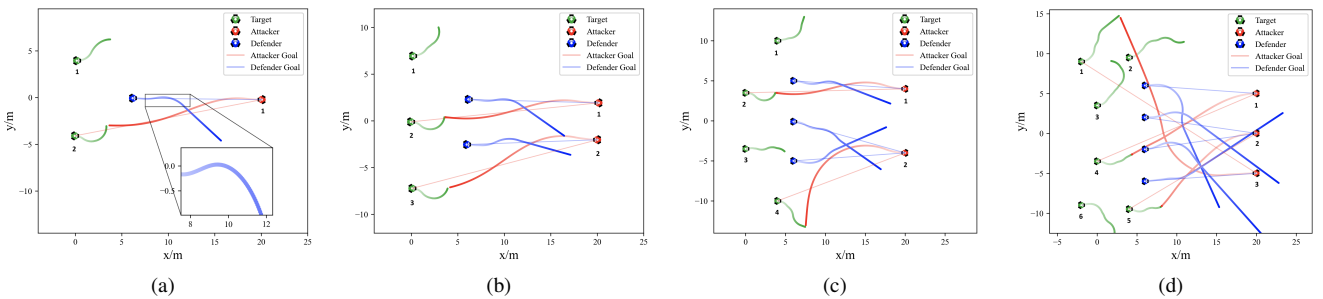


Fig. 3: Trajectories of different MR-TAD scenarios, with green representing Targets, red representing Attackers, and blue representing Defenders. (a) Scenario 2T-1A-1D. (b) Scenario 3T-2A-2D. (c) Scenario 4T-2A-3D. (d) Scenario 6T-3A-4D.

Fig. 4 illustrates the 2T-1A-1D scenario of a TAD game under real-world experiment. As can be seen, the Attacker initially selects a fake target and drives the Defender upward. At the right moment, the Attacker switches its belief to the true target, leaving the Defender with insufficient time to change direction and intercept. A more detailed depiction is provided in Fig. 3(a), where the Defender's trajectory is magnified, revealing a turn in its path caused by the fake target. The remaining experiments conducted on physical platforms are presented in the affiliated video.

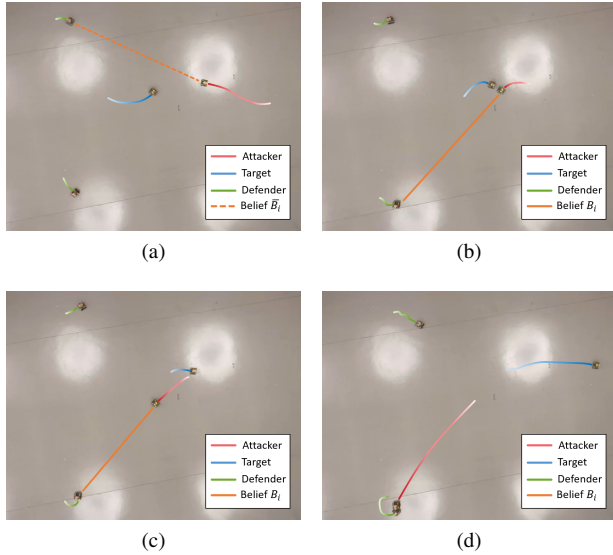


Fig. 4: Trajectory of real-world experiment on scenario 2T-1A-1D. (a) The Attacker selects a fake Target. (b) The timestep when Defender almost hits the Attacker. (c) The Attacker successfully penetrates the Defender. (d) The Attacker hits the true Target.

C. Deceptive Behavior Analysis

To evaluate the deceptive behavior of robots, we track the variations in Attackers' beliefs, as shown in Fig. 5. The vertical axis represents both the Target indices and Attackers' beliefs, while the horizontal axis denotes the time. We present only the time window before Attackers lock their beliefs. As observed in both scenarios, all Attackers successfully demonstrated effective deception by strategically switching their beliefs to evade Defender interception.

In the 3T-2A-2D scenario, Attacker 1 initially deceives its Defender by selecting Target 1 as a fake Target, then switches to Target 3 to create more space and evade Defender 1. Finally, it commits to its true belief, Target 2, and successfully reaches it. Similarly, Attacker 2 employs a deceptive strategy akin to that in the 2T-1A-1D scenario, by first selecting upper Targets as fake beliefs before ultimately reaching its true Target in the lower region. The 6T-3A-4D scenario can be analyzed in the same way. The most representative example of deception is observed in Attacker 3's strategy, where its true belief is Target 1, yet it initially selects the farthest Target, Target 6, as a fake belief to mislead the Defenders.

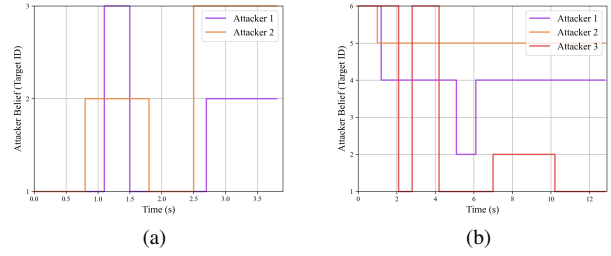


Fig. 5: Variation of Attackers' beliefs. (a) Scenario 3T-2A-2D. (b) Scenario 6T-3A-4D.

D. Convergence and Reward Analysis

The visualization of training rewards is shown in Fig. 6. We selected two scenarios and performed ablation tests by removing either R_{swt} or R_{pos} . As depicted in Fig. 6, the training reward achieves the highest convergence value and fastest convergence rate when both R_{swt} and R_{pos} are included, indicating that the deception-oriented reward enhances the effectiveness of deceptive behavior. When either R_{swt} or R_{pos} is removed, both the initial reward and the convergence value encounter a noticeable decline. However, in all cases, the training reward still converges properly, demonstrating the robustness of the proposed deception-based hierarchical decision-making framework.



Fig. 6: Training rewards of different scenarios. (a) Scenario 4T-2A-3D. (b) Scenario 6T-3A-4D.

The average training reward and its standard deviation are presented in Table II. The standard deviation of the mean reward across epochs quantifies reward fluctuation, reflecting the stability of the policy during training. Consistent with the previous analysis, the designed reward achieves the highest average value. Meanwhile, its standard deviation is generally lower than that of the other two cases, further indicating improved training stability.

E. Performance Analysis

Furthermore, to validate the effectiveness of deceptive strategies, we conducted a Monte Carlo test and evaluated the success rate under each scenario. Gaussian noise was added to the initial positions of all robots, and each scenario was simulated 100 times. Denoting the number of Attackers successfully reaching their Targets as N_{suc} , the success rate is

computed as $N_{\text{suc}}/(100 \times N_a)$. For comparison, we also executed the TAD algorithm without deception and calculated its corresponding success rate. The results are presented in Table III.

TABLE II: Average rewards in different scenarios

Scenarios	Average Reward	Average Reward without R_{swt}	Average Reward without R_{pos}
2T-1A-1D	153.38 $\pm$ 36.72	116.09 $\pm$ 49.85	85.92 $\pm$ 43.94
3T-2A-2D	189.31 $\pm$ 37.81	122.65 $\pm$ 45.50	103.10 $\pm$ 34.33
4T-2A-3D	250.48 $\pm$ 39.94	146.13 $\pm$ 43.78	134.38 $\pm$ 49.50
6T-3A-4D	307.72 $\pm$ 61.54	220.49 $\pm$ 62.33	196.14 $\pm$ 48.30

Table III illustrates that when Attackers do not employ deceptive strategies, they are more easily intercepted by Defenders. However, when deception is incorporated into the multi-robot TAD game, the Attackers' success rate increases significantly. In fact, the initial position configuration in our experiment inherently favors the Defenders as discussed in [12]. Therefore, deception and other cooperative strategies are essential for the Attackers to achieve success.

TABLE III: Success rates of deception in different scenarios

Scenarios	2T-1A-1D	3T-2A-2D	4T-2A-3D	6T-3A-4D
Success rate with deception	94.0%	87.0%	76.0%	71.7%
Success rate no deception	0.0%	1.5%	1.0%	1.0%

V. CONCLUSION AND FUTURE WORK

This article presents a hierarchical decision-making framework that adopts deception in a MR-TAD game. In the proposed framework, the high-level deception strategies are generated using deep reinforcement learning, while low-level control is implemented through optimal control and proportional navigation guidance. To facilitate the training of deceptive strategies, we designed a discrete action space, an observation space, and a deception-oriented reward function. Extensive experiments were conducted to validate the effectiveness of our approach. The results demonstrate that integrating deception into the game significantly enhances the Attackers' success rate in reaching their Targets.

For future work, we aim to conduct a more in-depth exploration of deception and its applications in other multi-agent tasks. Additionally, we will investigate alternative cooperative strategies in MR-TAD games.

ACKNOWLEDGMENT

The authors would like to thank Yingheng Liu for his generous support in establishing the mobile robot platform and providing indispensable technical assistance.

REFERENCES

- [1] M. Tan, S. Zhang, M. Stevens, D. Li, and E. J. Tan, "Antipredator defences in motion: animals reduce predation risks by concealing or misleading motion signals," *Biol. Rev.*, vol. 99, no. 3, pp. 778–796, 2024.
- [2] C. Kwon, W. Liu, and I. Hwang, "Security analysis for cyber-physical systems against stealthy deception attacks," in *2013 Amer. Control Conf.*, 2013, pp. 3344–3349.
- [3] A. R. Wagner and R. C. Arkin, "Acting deceptively: Providing robots with the capacity for deception," *Int. J. Social Robot.*, vol. 3, no. 1, pp. 5–26, Jan. 2011.
- [4] V. Comandur, T. R. Vechalapu, V. R. Makkapati, P. Tsiotras, and S. Hutchinson, "Desensitization and deception in differential games with asymmetric information," *Dyn. Games Appl.*, Jul. 2024.
- [5] M. Ornik and U. Topcu, "Deception in optimal control," in *2018 56th Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, Oct. 2018, pp. 821–828.
- [6] M. O. Karabag, M. Ornik, and U. Topcu, "Deception in supervisory control," *IEEE Trans. Autom. Control*, vol. 67, no. 2, pp. 738–753, Feb. 2022.
- [7] L. Huang and Q. Zhu, "A dynamic game framework for rational and persistent robot deception with an application to deceptive pursuit-evasion," *IEEE Trans. Automat. Sci. Eng.*, vol. 19, no. 4, pp. 2918–2932, Oct. 2022.
- [8] A. Asgharnia, H. M. Schwartz, and M. Atia, "Deception in the game of guarding multiple territories: A machine learning approach," in *2020 IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2020, pp. 381–388.
- [9] —, "Deception in a multi-agent adversarial game: The game of guarding several territories," in *2020 IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2020, pp. 1321–1327.
- [10] —, "Multi-invader multi-defender differential game using reinforcement learning," in *2022 IEEE Int. Conf. Fuzzy Syst. (FUZZ-IEEE)*, Jul. 2022, pp. 1–8.
- [11] R. L. Boyell, "Defending a moving target against missile or torpedo attack," *IEEE Trans. Aerosp. Electron. Syst.*, vol. AES-12, no. 4, pp. 522–526, 1976.
- [12] L. Liang, F. Deng, Z. Peng, X. Li, and W. Zha, "A differential game for cooperative target defense," *Automatica*, vol. 102, pp. 58–71, Apr. 2019.
- [13] L. Liang, F. Deng, M. Lu, and J. Chen, "Analysis of role switch for cooperative target defense differential game," *IEEE Trans. Autom. Control*, vol. 66, no. 2, pp. 902–909, Feb. 2021.
- [14] E. Garcia, D. W. Casbeer, and M. Pachter, "Design and analysis of state-feedback optimal strategies for the differential game of active defense," *IEEE Trans. Autom. Control*, vol. 64, no. 2, pp. 553–568, Feb. 2019.
- [15] F. Liu, X. Dong, Q. Li, and Z. Ren, "Cooperative differential games guidance laws for multiple attackers against an active defense target," *Chin. J. Aeronaut.*, vol. 35, no. 5, pp. 374–389, May 2022.
- [16] S. K. Singh, P. V. Reddy, and B. Vundurthy, "Study of multiple target defense differential games using receding horizon-based switching strategies," *IEEE Trans. Control Syst. Technol.*, vol. 30, no. 4, pp. 1403–1419, Jul. 2022.
- [17] M. Coon and D. Panagou, "Control strategies for multiplayer target-attacker-defender differential games with double integrator dynamics," in *2017 IEEE 56th Annu. Conf. Decis. Control (CDC)*, Dec. 2017, pp. 1496–1502.
- [18] Y. Yuan, J. Yang, Z. L. Yu, Y. Cheng, P. Jiao, and L. Hua, "Hierarchical goal-guided learning for the evasive maneuver of fixed-wing UAVs based on deep reinforcement learning," *J. Intell. Robot. Syst.*, vol. 109, no. 2, p. 43, Oct. 2023.
- [19] Z. Sun et al., "Multi-agent hierarchical policy gradient for air combat tactics emergence via self-play," *Eng. Appl. Artif. Intell.*, vol. 98, p. 104112, Feb. 2021.
- [20] W. Ou, B. Luo, B. Wang, and Y. Zhao, "Modular hierarchical reinforcement learning for multi-destination navigation in hybrid crowds," *Neural Netw.*, vol. 171, pp. 474–484, 2024.
- [21] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 24 611–24 624, 2022.
- [22] S. R. Kumar and T. Shima, "Cooperative nonlinear guidance strategies for aircraft defense," *J. Guid., Control, Dyn.*, vol. 40, no. 1, pp. 124–138, 2017.
- [23] R. Lowe, Y. Wu, and Tamar, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [24] A. Sherstinsky, "Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, Mar. 2020.